

Execution Guide: Transitioning to Llama 3.2 1B

To deploy your slang translator using the new Llama 3.2 1B configuration, follow these steps in order. This workflow ensures your data is correctly formatted for the Llama-specific chat template before training begins.

Step 1: Setup the Deployment Branch

Ensure you are working within your new `deployment/` directory or branch. This keeps your Mistral 7B backup safe while you optimize for low-VRAM environments.

Step 2: Install Additional Requirements

Llama 3.2 and the latest quantization techniques may require updated libraries. Run:

```
pip install --upgrade transformers datasets peft trl bitsandbytes accelerate
```

Step 3: Authenticate with Hugging Face

Llama 3.2 is a gated model. You must:

1. Request access on the [meta-llama/Llama-3.2-1B-Instruct](https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct) page.
2. In your terminal, run `huggingface-cli login` and enter your access token.
3. Alternatively, ensure your `HUGGINGFACE_HUB_TOKEN` is set in your environment variables.

Step 4: Prepare and Format the Data

Run the preprocessing and formatting scripts to generate the Llama-specific JSONL file.

```
# 1. Clean the raw data (standardize slang/formal pairs)
python preprocess_data.py

# 2. Convert CSV to Llama-formatted JSONL
# This uses the specific <|start_header_id|> tokens found in your Canvas file.
python deployment/format_data.py
```

Step 5: Execute Fine-Tuning

Run the training script. Because Llama 3.2 1B is much smaller, this will be significantly faster than your Mistral runs.

```
python deployment/fine_tune.py
```

- **VRAM Check:** This should only consume ~5-6GB during training and ~1GB during inference.

Step 6: Update the Inference Script

You will need to update your `infer.py` to point to the new `llama_translator` checkpoint and use the Llama 3.2 prompt template.

Step 7: Deployment to Streamlit/Vercel

- **Streamlit:** Upload your repository, ensuring `requirements.txt` is updated. Point your app to load the 1B model.
- **Vercel:** Since you cannot run the model directly, use the code in your `deployment` branch to create a backend API on a service like **Render** or **Groq**, and have Vercel act as your frontend.